

Privacy Act Statement for Recording

Privacy Act Statement (Record Meetings and Training Sessions)

Authority: The collection of this information is authorized under 5 U.S.C. 301 (*Departmental regulations*), 5 USC 552a (*Records maintained on individuals*); 15 U.S.C. 1512 (*Powers and duties of Department*), and 44 U.S.C. 2904 (General responsibilities for records management).

Purpose: Individual's permission is required for use of Artificial Intelligence (AI) in the collection, storage, reproduction, or summary of PII in any format, used for communications, outreach, Meet summaries, interviews, and dissemination of mission products intended to assist in NOAA's mission and utilize AI to expedite and improve NOAA's decision-making and data analysis.

Routine Uses: The information is used for the purpose set forth above and may be: forwarded to another NOAA or non-NOAA user account; shared among NOAA staff for work-related purposes, ingested into NOAA's AI applications for data analysis and learning models. Meet summaries and other AI products may be shared externally or displayed on NOAA websites and social media platforms, and as part of physical displays/exhibits. Disclosure of this information is permitted under the Privacy Act of 1974 (5 U.S.C. Section 552a) to be shared among Department staff for work-related purposes. Disclosure of this information is also subject to all of the published routine uses as identified in the Privacy Act System of Records Notice [\[Link\]](#), Employees Personnel Files Not Covered by Notices of Other Agencies, and [NOAA-11](#). Contact Information for Members of the Public Requesting or Providing Information Related to NOAA's Mission.

Disclosure: Voluntary; by joining and participating in the meeting consent is being given to the use of their PII in the AI application.

Please keep your cameras off during recording! PII

Adding Idealized Test Cases to the UFS Hierarchical System Development (HSD)

Framework

NOAA/EPIC Instructors: Natalie Perlin, Kris Booker (tech support)

Assistant Instructors: Joshua Kublnik, Priya Pillai

Training materials credits: Ratko Vasic, Cameron Book

20 July 2026 (College Park, MD)



Agenda

- Introduction and communication
- Logging in to your HPC sandbox
- Downloading the UFS Weather-Model/HSD Code
- UFS Hierarchical System Development (HSD) Framework
- Directory structure
- Running HSD Tests
- Adding a new case
 - Steps for a general case
 - Follow Aquaplanet as example
- Exercise 1
- Exercise 2
- Q&A

NOAA EPIC Instructors

- Natalie Perlin - Lead, in-person: natalie.perlin@noaa.gov
- Kris Booker (*) - Assistant, virtual: kristopher.booker@noaa.gov
- Joshua Kublnik - Assistant, in-person: joshua.kublnick@noaa.gov
- Priya Pillai - Assistant, in-person: priya.pillai@noaa.gov

Use Slack channel **#uifcw26-training-2-adding-idealized-test-cases** for any questions or communication, or DM to a particular instructor

(*) - Provisioning compute resources, SSH connection issues etc.



Communications and SSH Login

Recap of pre-training actions:

- **Slack Workspace** to join with your registered e-mail:
<https://join.slack.com/t/epicworkshops-pza9734/signup>
- Join the following Slack channels:
 - **#uifcw26-training-2-adding-idealized-test-cases**
-for general communication between participants and instructors
 - **#public-ssh-keys-hsd-training**
- upload your public SSH keys for SSH connection
- verify your username for the login: <firstname>.<lastname>
- Prior to the HSD training, run the following command from PowerShell or command prompt (Windows) or Mac terminal Application to test your connection:

```
ssh -i <private-key-file> test@jump2.epic.noaa.gov
```
- Connect to your HPC sandbox during Training:

```
ssh <firstname>.<lastname>@jump2.epic.noaa.gov
```



Connect via SSH to your HPC environment

- `ssh <firstname>.<lastname>@jump2.epic.noaa.gov`
- `cd /scratch`
- `mkdir <username>`
- `cd <username>`

where `<firstname>.<lastname>` is your verified username for the remote login,

`<username>` is a local directory name of your choice.

(Send a message via Slack to **Kris Booker** in case of SSH connection issues!)

Reference for SSH keys generation and HSD Training Session commands:

<https://github.com/NOAA-EPIC/uifcw2026/blob/main/HSD-training/README.md>



Quick Launch of the HSD Aquaplanet Test

- `cd /scratch/<username>`
- `git clone --recursive -b feature/2026-HSD-training`
<https://github.com/NOAA-EPIC/ufs-weather-model.git>
- `cd ufs-weather-model`
- `export SRC_DIR=$PWD`
- `cd tests-dev`

A convention: commands highlighted in gray indicate as a single long line

- `./ufs_test.sh -a ubuntu -s -c -k -r -n "aquaplanet intel"`

Account name

Identifier for
HSD tests

Mimic rt.sh "create new
results" mode – no
baseline comparison

Keep the results
directory

Test and compiler names

Use rocoto
workflow manager



Acknowledgements



- The development and integration of HSD and its test cases would not have been possible without the collaboration and support of our NOAA EPIC partners:
 - NOAA National Weather Service (NWS) Office of Modeling and Development (OMD)
 - Developmental Testbed Center (DTC)
 - University of Michigan
 - NOAA Physical Sciences Laboratory (PSL)



Introduction to UFS Hierarchical System Development (HSD)

HSD is a **model-development strategy** [Ek et al. 2019*, Sun et al., 2023**]

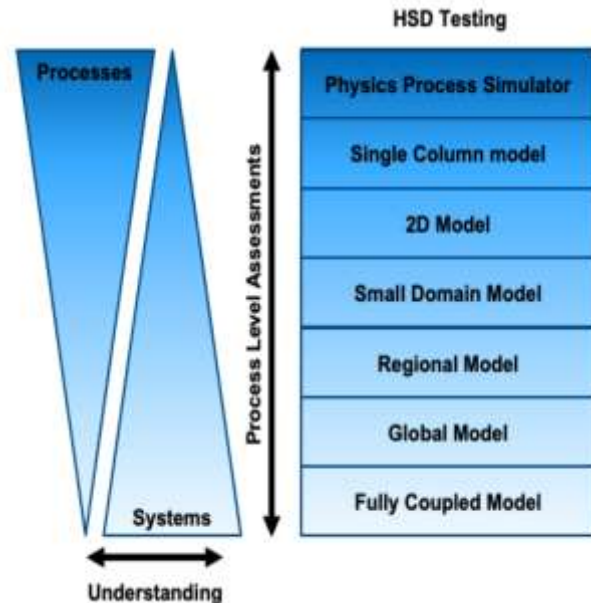
- test new science
- test multiple levels of complexity
- Starts with simple model configurations, for example:
 - idealized cases
 - single-column models
 - process-oriented models
 - single-component only (ATM, OCN, LND)
- Progresses toward fully-coupled and operationally realistic systems
- Process-level evaluation at every step
- Allows to understand both processes and systems
- Entry point for users to test innovations/code/workflow changes to the UFS

HSD goals: Integrate academic work into operational framework (UFS-WM)

<https://epic.noaa.gov/get-code/hierarchical-system-development/>

(*) <https://ufs.epic.noaa.gov/2019/09/hierarchical-system-development-for-the-ufs/>

(**) <https://dtcenter.org/sites/default/files/page/white-paper-ufs-hsd.pdf>



What HSD Does: Four Axes of Testing

- Each axis gives developers a different lever for isolating a model problem before testing it in the full system



Figure adapted from: <https://dtcenter.org/sites/default/files/page/white-paper-ufs-hsd.pdf>

From Concept to Capability: HSD in UFS Today

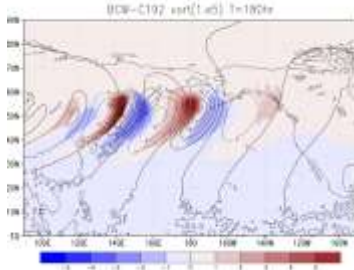
Existing tools already implement pieces of the hierarchy — EPIC is now packaging them into a shared HSD framework:

EPIC HSD Test-Case Releases		
●	Nov 2024	July 2020 CAPE case <i>low-CAPE bias in GFS; C48, 24-hr forecast</i>
●	Nov 2024	Baroclinic instability wave <i>Idealized dry dynamical-core test; C192</i>
●	Jun 2025	Idealized regional tropical cyclone <i>with HAFS group; 4-km, 5-day forecast</i>
●	Feb 2026	Aquaplanet case <i>with NOAA PSL; ocean-only surface, C48</i>

HSD workflow currently configured for the following platforms:

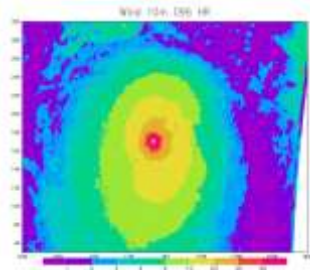


Currently available HSD cases:



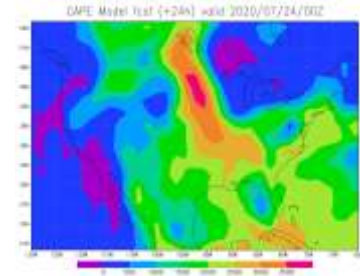
Dry baroclinic wave

- C192 resolution, 127 vertical levels
- FV3 dynamical core only (no physics)



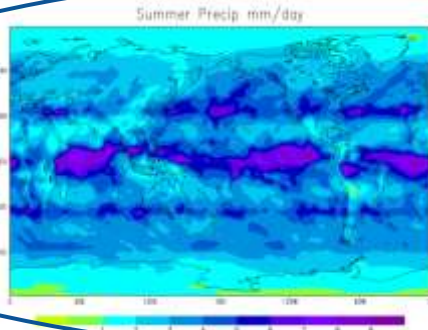
Tropical cyclone

- 4-km resolution; 81 vertical levels
- FV3_HAFS_v1_thompson_nonsst CCM physics



CAPE 2020

- C48 resolution, 127 vertical levels
- FV3 GFS V16 CCM physics



Aquaplanet

- C48 resolution, 127 vertical levels
- Ocean-covered planet (ATM) 90-day spinup
- FV3_GFS_v17_p8_ugwv1 physics suite

see <https://epic.noaa.gov/get-code/hierarchical-system-development/>

UFS Weather Model directory structure:

/scratch/<username>/ufs-weather-model/

- AQM
- CDEPS-interface
- CICE-interface
- CMEPS-interface
- CMakeLists.txt
- CMakeModules
- GOCART
- LM4-driver
- MOM6-interface
- NOAHMP-interface
- README.md
- UFSATM
- WW3
- build.sh
- ci
- cmake
- doc
- driver
- fire_behavior
- modulefiles
- stochastic_physics
- tests
- tests-dev
- ufs_tracing

HSD workflow is located in this directory:



HSD workflow directory structure:

```
└─ tests-dev
   │
   └─ test_cases
      │
      ├── diag_table
      │
      ├── field_table
      │
      ├── parm
      │
      ├── tests
      │
      ├── utils
      │
      └─ exp_conf
   └─ machine_config
   └─ logs
```

(similarity with `./ufs-weather-model/tests:`)

```
└─ tests
   │
   ├── tests
   │
   ├── fv3_conf
   │
   ├── parm
   │   │
   │   └─ field_table
   │       │
   │       └─ diag_table
   └─ logs
```

Utilizing UFS HSD: Framework Overview

HSD workflow in **ufs-weather-model/tests-dev/** contains:

- driver scripts (*bash/python: *.sh, *.py*)
- workflow/resource configuration files
- model tests (RT) configuration files (**./tests**)
- HSD tests configuration files, namelists (**./test_cases/parm** , **./exp_conf**)
- HSD-specific override tables (**./test_cases/diag_table**, **./test_cases/field_table**)
- plotting utilities (**./test_cases/utls**)

Top-level scripts and configuration files:

- **ufs_test.sh**: the main driver script to compile/run tests; calls other bash & Python configuration scripts
- **baseline_setup.yaml**: machine-specific location of input data, output paths, job scheduler configuration
- **ufs_test.yaml**: contains available test list, compile options



HSD Tests Input and Configuration Files

- **./tests-dev/baseline_setup.yaml** - machine-specific location of pre-staged climatology and model input data, output paths, job scheduler configurations
- **./tests/default_vars.sh** - machine-specific variables, including **TPN** (tasks-per-node), horizontal grid decomposition details for certain resolutions and model configurations, component-specific task details.
- **./tests-dev/test_cases/tests/<case_name>** - forecast details (start, stop, output frequency), grid resolution, restart option, grid sizes, CCM3 physics suite, output grid details, write task details, names of other input files:
 - **./tests-dev/test_cases/exp_conf/<case_name>.IN** - (e.g., **aquaplanet.IN**)
 - data staging script for the run, resolution-dependent configurations
 - **./tests-dev/test_cases/parm/<case_name>.nml.IN** (e.g., **aquaplanet.nml.IN**)
 - namelists for components, physics/microphysics, schemes
 - **./tests-dev/test_cases/parm/model_configure_<cn>.IN** (e.g., **model_configure_aq.IN**)
 - model configuration
 - wall-clock for the test
 - land-surface data: sea-land mask, soil moisture
 - TPN (overrides defaults from **./tests/default_vars.sh**)
- **./tests-dev/machine-config/machine_<platform>.config**
- **./tests/bl_date.conf** - staged baseline date stamp

Main Driver for the HSD Workflow

The main driver in `/scratch/<username>/ufs-weather-model/tests-dev/` directory:

`./ufs_test.sh`

Options:

- a <account> to use on for HPC queue
- b create new baselines only for tests listed in <file>
- c create new baseline results
- d delete run directories that are not used by other tests
- e use ecFlow workflow manager (this option is not fully functional yet)
- h display this help
- k keep run directory after ufs_test.sh is completed
- l runs test specified in <file>
- m compare against new baseline results
- n run single test <name>
- o compile only, skip tests
- r use Rocoto workflow manager
- w for weekly_test, skip comparing baseline results
- s for use tests-dev, symlink sharable tests scripts



General Approach to Running an HSD Test Case

HSD workflow configured for the following platforms:

ursa, hera, orion, hercules, gaeac6, derecho, noaacloud, awspc

From your `./ufs-weather-model/tests-dev/` directory, run the following:

```
./ufs_test.sh -a ubuntu -s -c -k -r -n "<test-name> intel"
```

Replace `<test-name>` with the available case names(*) listed below:

- **aquaplanet**
- 2020_CAPE
- baroclinic_wave
- tropical_cyclone

(*) Note that the cluster configurations for the current Training Session support only "aquaplanet" case



Running the HSD test case

Results:

Under `./ufs-weather-model/tests-dev/run_dir/<test-name_compiler>/`

Link to run directory is `./run_dir -> your-run-dir-full-path`

Run directory contains:

- Fixed files (climatology, orography etc)
- Initial (and boundary) conditions
- Namelists
- Modulefiles, executable, job card etc.

Adding a new HSD test

General steps in creating a new test (for information only!)

- 1) Start from single regression test of choice
- 2) Save run directory containing all necessary ICs, BCs, climatology, topography, ...
- 3) Make changes in namelists i.e. **input.nml** and **model_configure**
- 4) Make changes (if necessary) for particular experiment in input files (ICs, BCs, ...)
- 5) Add changes in tests-dev directory:
 - a) **ufs_test.sh** and **ufs_test.yaml**
 - b) **~/test_cases/exp_conf/new-test.IN**
 - c) **~/test_cases/parm/new-test.nml.IN**
 - d) **~/test_cases/tests/new-test**



Note: we do not repeat these steps during the Training Session!

Adding new HSD test (2)

Start from a list of tests in `./tests/rt.conf` with available model RTs, pick the one closest to your needs.

(Test control_c48 for *aquaplanet*):

RUN	control_CubedSphereGrid		baseline	
RUN	control_CubedSphereGrid_parallel	- noaacloud	baseline	
RUN	control_latlon		baseline	
RUN	control_wrtGauss_netcdf_parallel		baseline	
RUN	<u>control_c48</u>		baseline	
RUN	control_c48.v2.sfc		baseline	
RUN	control_c48_lnd_iau		baseline	
RUN	control_c192	- noaacloud	baseline	
RUN	control_c384		baseline	
RUN	control_c384gdas	- noaacloud	baseline	
RUN	control_stochy		baseline	

Note: we do not repeat these steps during the Training Session!

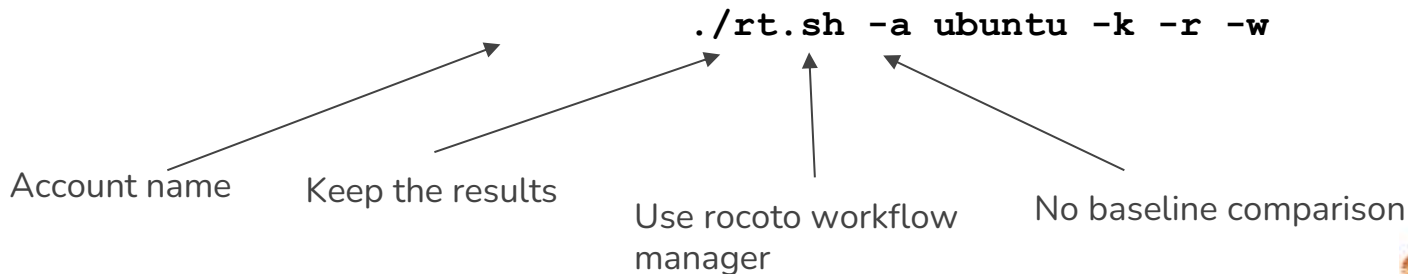
Adding new HSD test (3)

- **Edit `rt.conf`:** Leave two lines, one for compilation, and one for running the test.

Choose appropriate CCPP package (FV3_GFS_v17_p8_ugwpv1).

```
COMPILE | atm_dyn32 | intel | -DAPP=ATM -DCCPP_SUITES=FV3_GFS_v17_p8_ugwpv1 -D32BIT=ON | | fv3 |  
RUN | control_c48 | | baseline |
```

- **Run `rt.sh`:**



Note: we do not repeat these steps during the Training Session!

Adding new HSD test (4)

Run directory `./tests/run_dir/control_c48_intel/` contains all the required input files:

```
|— CCN_ACTIVATE.BIN
|— INPUT
|   |— C48_grid.tilex.nc
|   |— C48_mosaic.nc
|   |— gfs_ctrl.nc
|   |— gfs_data.tileX.nc
|   |— grid_spec.nc
|   |— oro_data.tileX.nc
|   |— sfc_data.tileX.nc
|   |— sfc_inc.tileX.nc
|— IMS-
NIC.blended.ice.monthly.clim.grb
|— RTGSST.1982.2012.monthly.clim.grb
|— diag_table
|— field_table
|— freezeH2O.dat
|— modulefiles
|   |— modules.fv3.lua
|   |— ufs_common.lua
|— fv3.exe
|— job_card
|— global_glacier.2x2.grb
|— global_maxice.2x2.grb
|— global_slmask.t62.192.94.grb
|— global_snoclim.1.875.grb
|— input.nml
|— model_configure
|— module-setup.sh
|— sfc_emissivity_idx.txt
|— solarconstant_noaa_an.txt
```



Note: we do not repeat these steps during the Training Session!

Climatology and fixed files:

```
|— CCN_ACTIVATE.BIN
|— INPUT
|   |— C48_grid.tilex.nc
|   |— C48_mosaic.nc
|   |— gfs_ctrl.nc
|   |— gfs_data.tileX.nc
|   |— grid_spec.nc
|   |— oro_data.tileX.nc
|   |— sfc_data.tileX.nc
|   |— sfc_inc.tileX.nc
|— IMS-NIC.blended.ice.monthly.clim.grb
|— RTGSST.1982.2012.monthly.clim.grb
|— diag_table
|— field_table
|— freezeH2O.dat

|— modulefiles
|   |— modules.fv3.lua
|   |— ufs_common.lua
|— fv3.exe
|— job_card
|— global_glacier.2x2.grb
|— global_maxice.2x2.grb
|— global_slmask.t62.192.94.grb
|— global_snoclim.1.875.grb
|— input.nml
|— model_configure
|— module-setup.sh
|— sfc_emissivity_idx.txt
|— solarconstant_noaa_an.txt
```

Note: we do not repeat these steps during the Training Session!

Initial and lateral boundary conditions:

- CCN_ACTIVATE.BIN
- INPUT
 - C48_grid.tilex.nc
 - C48_mosaic.nc
 - gfs_ctrl.nc
 - gfs_data.tileX.nc
 - grid_spec.nc
 - oro_data.tileX.nc
 - sfc_data.tileX.nc
 - sfc_inc.tileX.nc
- IMS-NIC.blended.ice.monthly.clim.grb
- RTGSST.1982.2012.monthly.clim.grb
- diag_table
- field_table
- freezeH2O.dat

- modulefiles
 - modules.fv3.lua
 - ufs_common.lua
- fv3.exe
- job_card
- global_glacier.2x2.grb
- global_maxice.2x2.grb
- global_slmask.t62.192.94.grb
- global_snoclim.1.875.grb
- input.nml
- model_configure
- module-setup.sh
- sfc_emissivity_idx.txt
- solarconstant_noaa_an.txt

Note: we do not repeat these steps during the Training Session!

Job card, executables and modulefiles:

- CCN_ACTIVATE.BIN
- INPUT
 - C48_grid.tilex.nc
 - C48_mosaic.nc
 - gfs_ctrl.nc
 - gfs_data.tileX.nc
 - grid_spec.nc
 - oro_data.tileX.nc
 - sfc_data.tileX.nc
 - sfc_inc.tileX.nc
- IMS-NIC.blended.ice.monthly.clim.grb
- RTGSST.1982.2012.monthly.clim.grb
- diag_table
- field_table
- freezeH2O.dat

- modulefiles
 - modules.fv3.lua
 - ufs_common.lua
- fv3.exe
- job_card
- global_glacier.2x2.grb
- global_maxice.2x2.grb
- global_slmask.t62.192.94.grb
- global_snoclim.1.875.grb
- input.nml
- model_configure
- module-setup.sh
- sfc_emissivity_idx.txt
- solarconstant_noaa_an.txt

Namelists:

Most likely candidates
for making changes to
create new test

- CCN_ACTIVATE.BIN
- INPUT
 - C48_grid.tilex.nc
 - C48_mosaic.nc
 - gfs_ctrl.nc
 - gfs_data.tileX.nc
 - grid_spec.nc
 - oro_data.tileX.nc
 - sfc_data.tileX.nc
 - sfc_inc.tileX.nc
- IMS-NIC.blended.ice.monthly.clim.grb
- RTGSST.1982.2012.monthly.clim.grb
- diag_table
- field_table
- freezeH2O.dat

- modulefiles
 - modules.fv3.lua
 - ufs_common.lua
- fv3.exe
- job_card
- global_glacier.2x2.grb
- global_maxice.2x2.grb
- global_slmask.t62.192.94.grb
- global_snoclim.1.875.grb
- input.nml
- model_configure
- module-setup.sh
- sfc_emissivity_idx.txt
- solarconstant_noaa_an.txt



Note: we do not repeat these steps during the Training Session!

Example from model_configure:

```
start_year:      2021
start_month:     03
start_day:       22
start_hour:      06
start_minute:    0
start_second:    0
nhours_fcst:     24
fhrot:           0
```

```
dt_atmos:        720
restart_interval: 0
restart_fh:
```

```
use_saved_routehandles: .false.
quilting:               .true.
quilting_restart:       .true.
write_groups:            1
write_tasks_per_group:   2
itasks:                  1
```

Start date

Forecast length

Timestep

Note: we do not repeat these steps during the Training Session!

Example from input.nml:

```
&atmos_model_nml  
  blocksize = 32  
  chksum_debug = .false.  
  dycore_only = .false.  
  ccpp_suite = 'FV3_GFS_v17_p8_uhwpv1'  
/
```

Physics suite from
CCPP package

Resolution

```
&fv_core_nml  
  layout = 1,1  
  io_layout = 1,1  
  npz = 49  
  npy = 49  
  ntils = 6  
  npz = 127
```

Number of
vertical levels

...

Note: we do not repeat these steps during the Training Session!

Adding new HSD test Aquaplanet test as an example

The Aquaplanet Test Case is an idealized atmosphere-only configuration used to study weather and climate dynamics. By eliminating land, topography, and sea-ice, the Earth's surface is modeled as an all-water covered globe with symmetric Sea Surface Temperatures (SSTs). This isolates key ocean-atmosphere interactions.

Detailed info [here](#) (5.4.5.1 - 5.4.4.17) and <https://epic.noaa.gov/aquaplanet-test-case/>.



Note: we do not repeat these steps during the Training Session!

1.Clone repositories:

UFS Weather Model:

```
git clone --recursive https://github.com/ufs-community/ufs-weather-model.git
```

UFS_UTILS:

```
git clone --recursive https://github.com/ufs-community/UFS_UTILS.git
```

Aquaplanet tools:

```
git clone https://github.com/NOAA-EPIC/Aquaplanet
```



Note: we do not repeat these steps during the Training Session!

2.Build the model:

UFS Weather Model:

- `cd ~/ufs-weather-model/tests/`

Edit `rt.conf` to include only the following line

```
COMPILE | atm_dyn32 | intel | -DAPP=ATM -DCCPP_SUITES=FV3_GFS_v17_p8_ugwpv1  
-D32BIT=ON | | fv3 | RUN | control_c48 | | baseline |
```

Execute the regression test to compile the model and create a baseline run directory:

- `./rt.sh -a ubuntu -k -w`



Note: we do not repeat these steps during the Training Session!

3.Build the UFS_UTILS:

Compile UFS_UTILS:

- `cd ${AP_EXPT}/UFS_UTILS/`
- `module purge`
- `module use $PWD/modulefiles`
- `module load build.<platform>.intellvm`
- `./build_all.sh`

Get fixed fields as shown in [Docs](#).



Note: we do not repeat these steps during the Training Session!

4. Generate ICs using UFS_UTILS:

Following instructions in [Docs](#):

Go to gdas_init utility directory in UFS_UTILS:

```
cd ~/UFS_UTILS//util/gdas_init/
```

Edit the config file

Edit the driver script: `driver.<platform>.sh`

Run the driver script: `./driver.<platform>.sh`



Note: we do not repeat these steps during the Training Session!

5. Create SST Profile Using Aquaplanet Tools

Navigate to the SST profile tool directory:

```
cd ~/Aquaplanet/sst-profile/
```

Copy the SST climatology file from your UFS Weather Model run directory.

Compile and run the tool:

```
./compile.sh
```

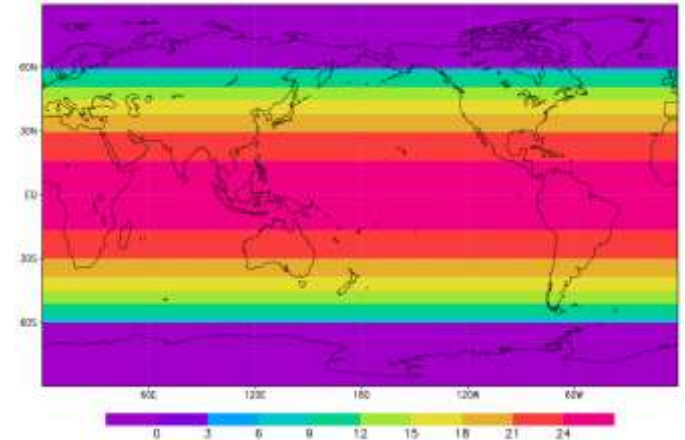
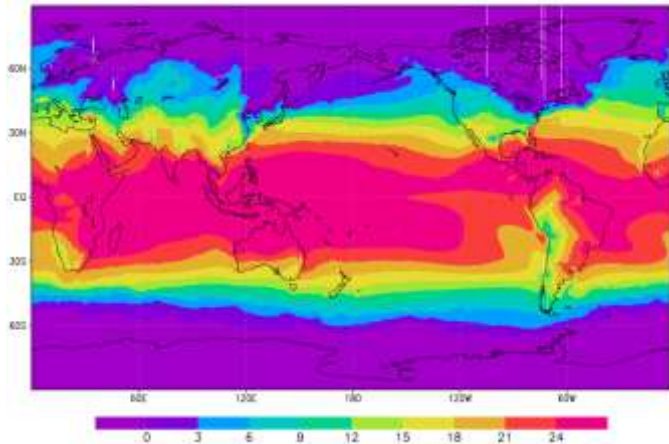
```
./sst-profile.x
```

Copy the modified file back to your run directory.



Note: we do not repeat these steps during the Training Session!

5. Create SST Profile Using Aquaplanet Tools:



6.Edit Global Ice Data:

Navigate to the glacier tool directory:

```
cd ~/Aquaplanet/glacier/
```

Copy the glacier file from your UFS Weather Model run directory.

Compile and run the tool:

```
./compile.sh
```

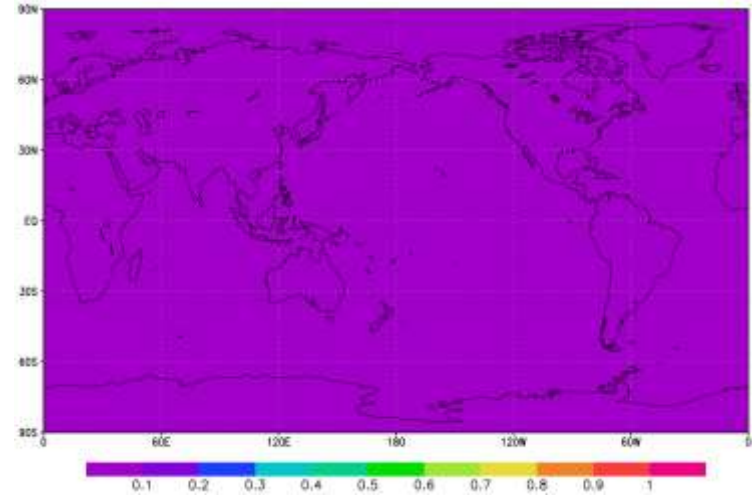
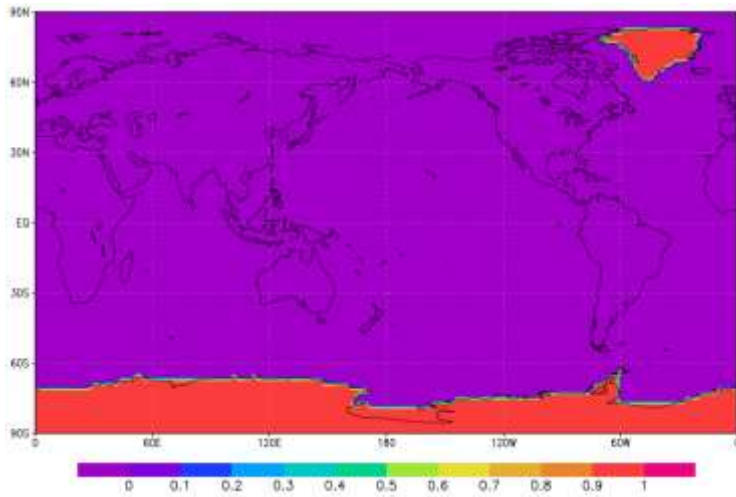
```
./glacier.x
```

Copy the modified file back to your run directory.



Note: we do not repeat these steps during the Training Session!

6.Edit Global Ice Data:



7.Edit Monthly Ice Data:

Navigate to the ice monthly tool directory:

```
cd ~/Aquaplanet/ice-monthly/
```

Copy the ice monthly file from your UFS Weather Model run directory.

Compile and run the tool:

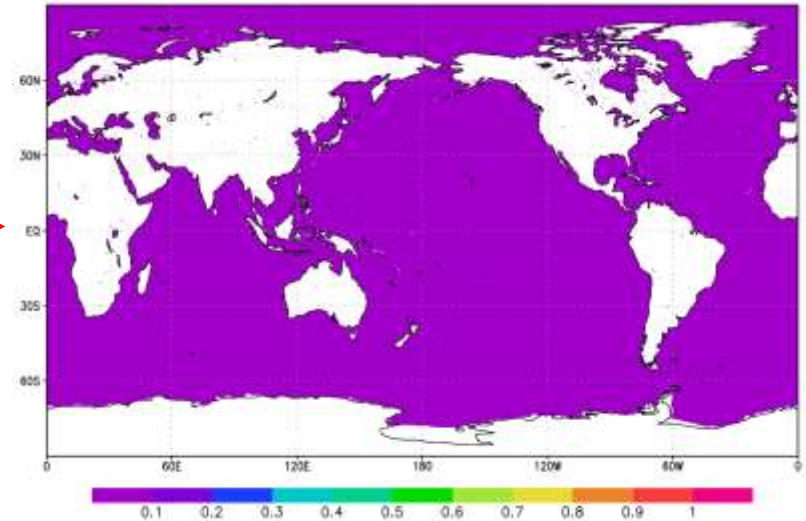
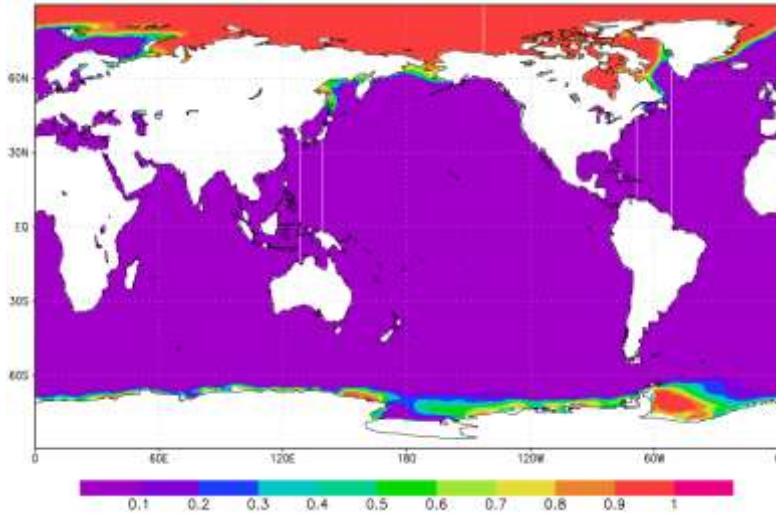
```
./compile.sh  
./ice-monthly.x
```

Copy the modified file back to your run directory.



Note: we do not repeat these steps during the Training Session!

7.Edit Monthly Ice Data:



Note: we do not repeat these steps during the Training Session!

8. Edit Sea-Land mask Data:

Navigate to the sea-land mask tool directory:

```
cd ~/Aquaplanet/slmask/
```

Copy the sea-land mask file from your UFS Weather Model run directory.

Compile and run the tool:

```
./compile.sh
```

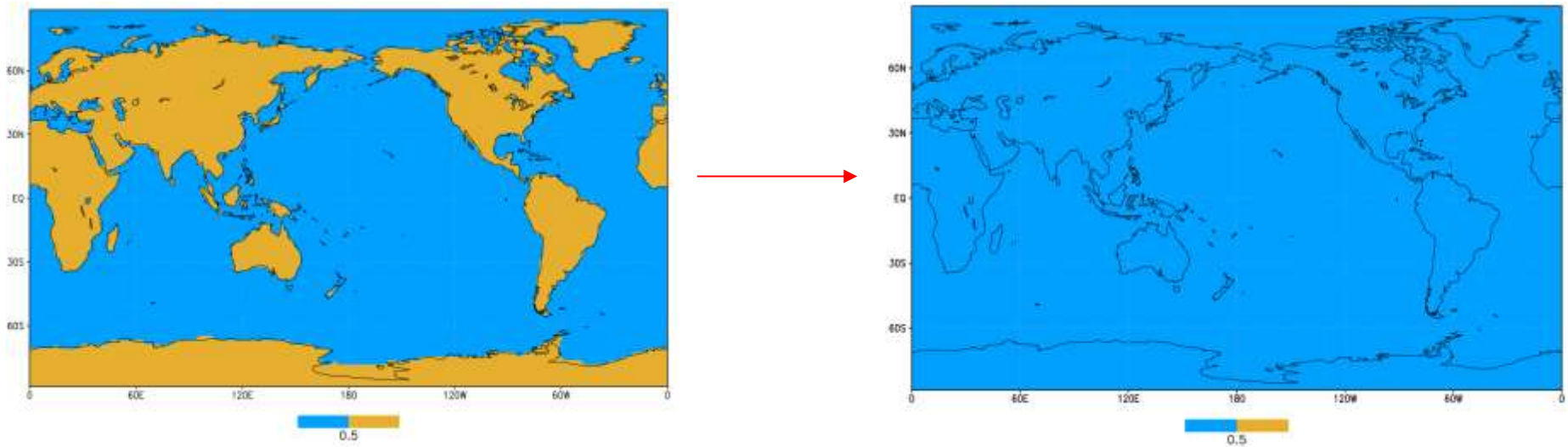
```
./slmask.x
```

Copy the modified file back to your run directory.



Note: we do not repeat these steps during the Training Session!

8.Edit Sea-Land mask Data:



9.Setup Atmospheric Profiles:

Navigate to the atmospheric profile tool directory:

```
cd ~/Aquaplanet/atmos-profile/
```

Copy the file with upper atmospheric fields from your UFS_UTILS directory (`gfs.t06z.atman1.nc`).

Compile and run the tool:

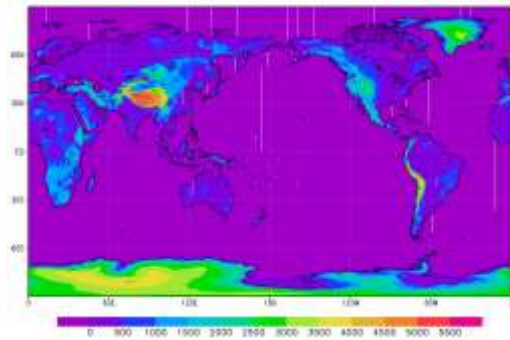
```
./compile.sh
```

```
./profile.x
```

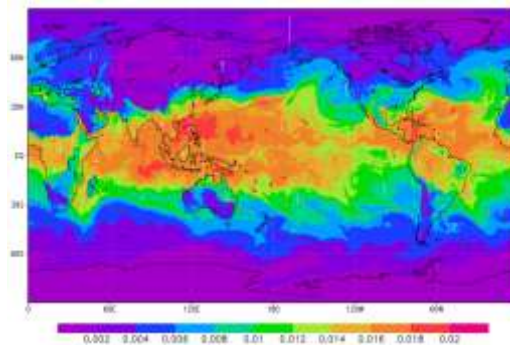
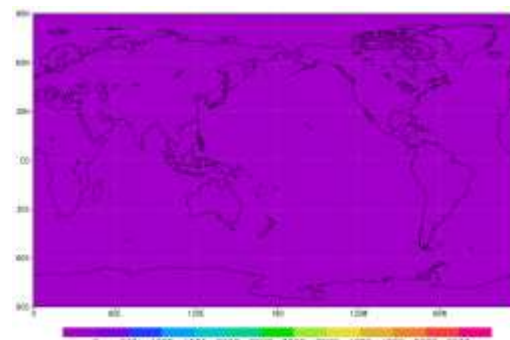
Copy the modified file back to original directory.



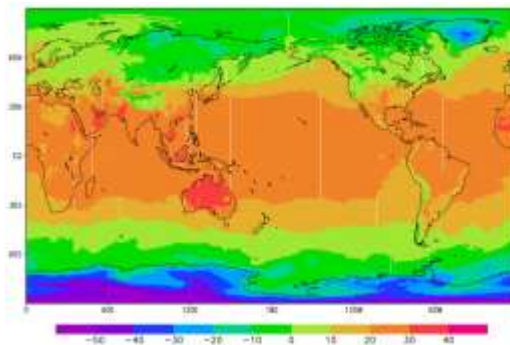
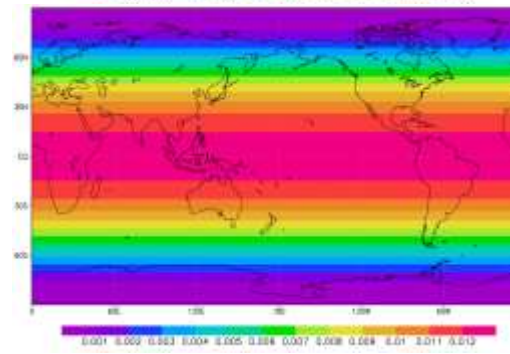
Note: we do not repeat these steps during the Training Session!



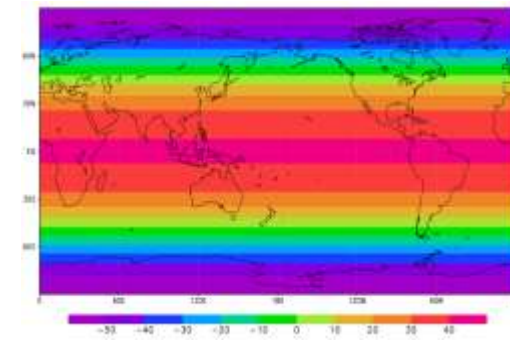
Geopotential
height



Specific
humidity



Temperature



10.Setup Surface Profiles:

Navigate to the surface profile tool directory:

```
cd ~/Aquaplanet/sfc-profile/
```

Copy the file with surface fields from your UFS_UTILS directory (`gfs.t06z.sfcan1.nc`).

Compile and run the tool:

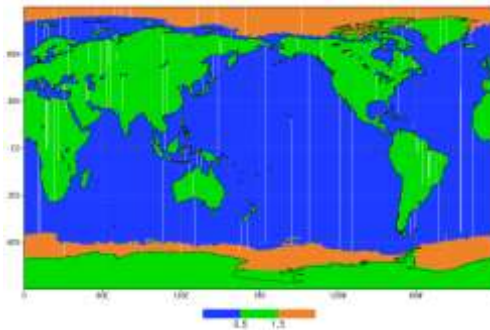
```
./compile.sh
```

```
./profile.x
```

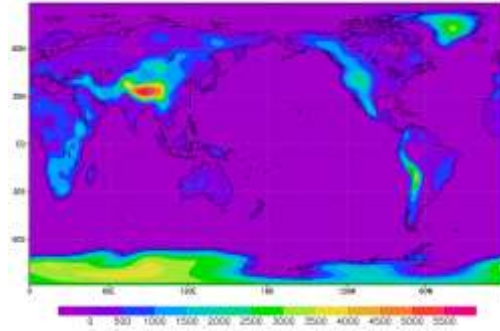
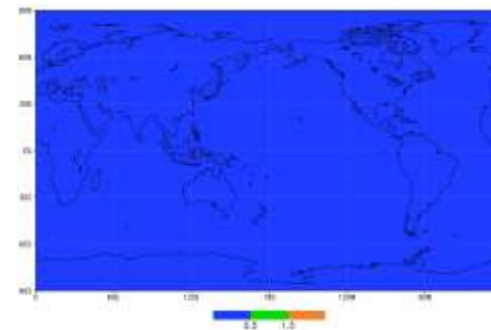
Copy the modified file back to original directory.



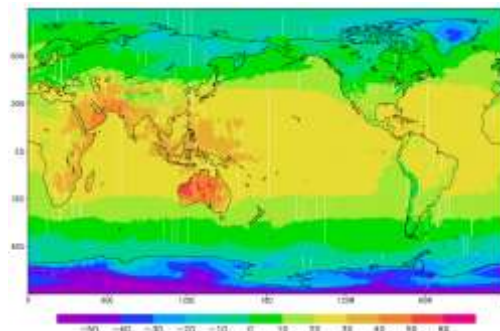
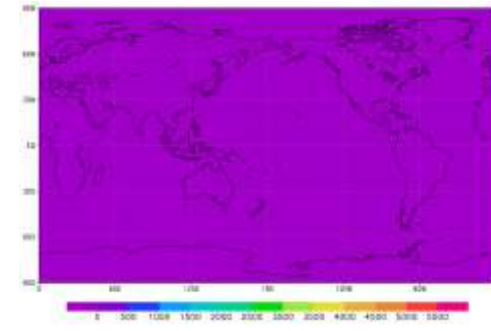
Note: we do not repeat these steps during the Training Session!



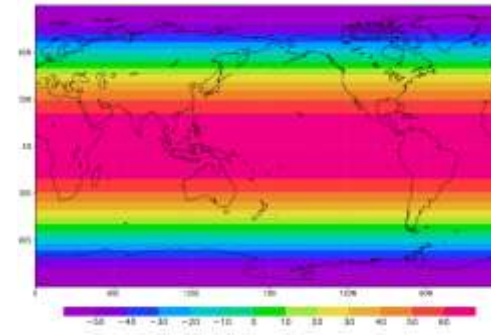
Sea-Land-Ice Mask



Orography



Surface Temperature



11.Prepare Orography Fields:

Navigate to the orography profile tool directory:

```
cd ~/Aquaplanet/orog-profile/
```

Compile and run the tool:

```
./compile.sh
```

```
./patch.x
```

This will modify the orography files in place to set land mask to ocean and height to zero.



Note: we do not repeat these steps during the Training Session!

12. Regenerate Initial Conditions:

With fixed fields and climatology files prepared, we run chgres (UFS_UTILS's main program) again.

- Go to gdas_init utility directory in UFS_UTILS:

```
cd ~/UFS_UTILS/util/gdas_init/
```

- Remove old ICs.
- In the config file, set EXTRACT_DATA to no since data is already extracted.
- Run the driver script: `./driver.<platform>.sh`
- We just regenerated ICs with new orography, sea-land mask, ice, SST, surface and upper atmosphere fields.



Note: we do not repeat these steps during the Training Session!

13.Store new initial conditions:

With fixed fields and climatology files prepared, we run **chgres** (UFS_UTILS's main program) again.

- Go to the run directory, input files subdirectory:

```
cd ~/ufs-weather-model/tests/run_dir/control_c48_intel/INPUT/
```

- Copy new initial conditions:

```
cp ~/UFS_UTILS/output/gfs.20251015/06/model/atmos/input/* .
```

- Copy modified orography files to the run's input directory:

```
cp ~/UFS_UTILS/fix/orog/C48/C48.mx500_oro_data.tile*.nc .
```



Note: we do not repeat these steps during the Training Session!

14.Changes in the source code:

This is time for making changes in the code. We can make changes to the model and/or CCpp physics package. Details in the [docs](#).

- Go to UFS WM directory: `cd ~/ufs-weather-model/`
- In this case we edited `vim UFSATM/ccpp/physics/physics/Radiation/radiation_astronomy.f` file to set solar zenith angle to zero:

```
! --- ... calculate sun-earth distance adjustment factor appropriate to date
      solcon = solc0 / (r1*r1)
! AQUAPLANET TEST CASE
      solcon=1370.
      dlt=0.
! AQUAPLANET TEST CASE

      slag  = sollag
      sdec  = sindec
      cdec  = cosdec
```

- Note - in final version, we are doing it automatically in `~/ufs-weather-model/tests-dev/ufs_test.sh`



Note: we do not repeat these steps during the Training Session!

15. Spinup Initial States:

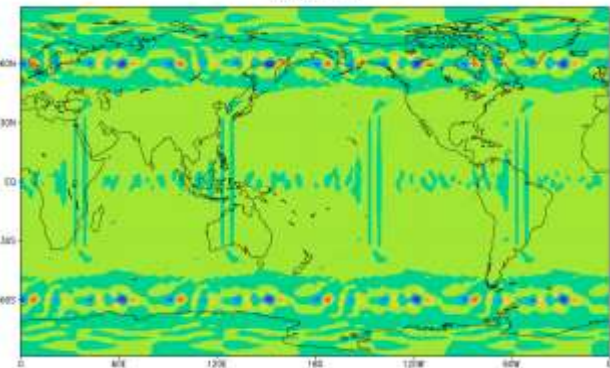
Since we start from idealized initial conditions, in the aquaplanet simulation we did a 90-day spin-up period to reach equilibrium.

Here are some examples of variables during the spin up period: zonal wind in high atmosphere, temperature and precipitation.

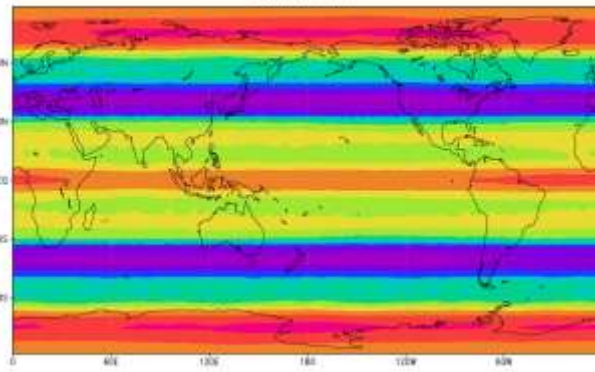


Note: we do not repeat these steps during the Training Session!

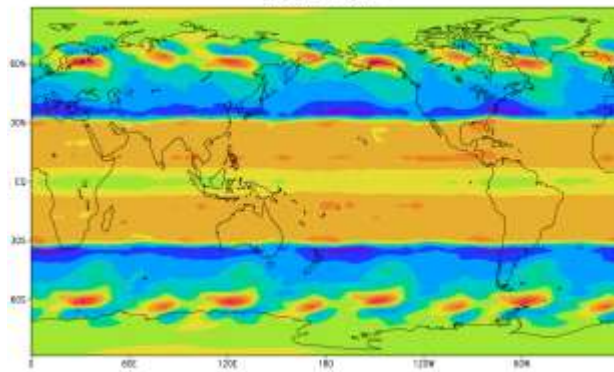
UGRD 0hr



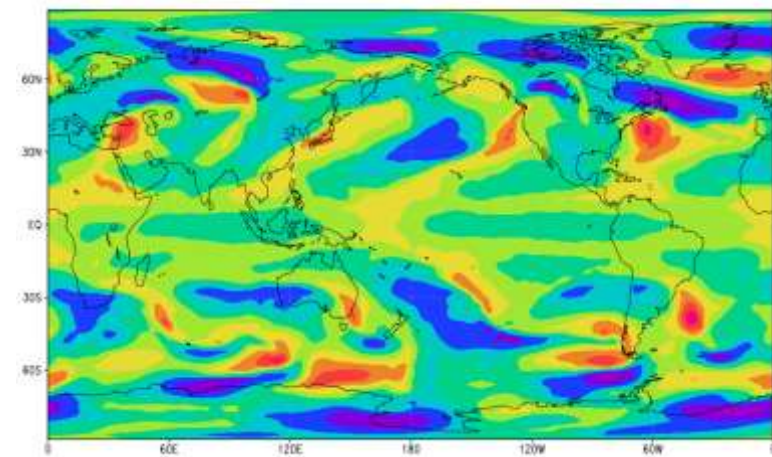
UGRD 24hr



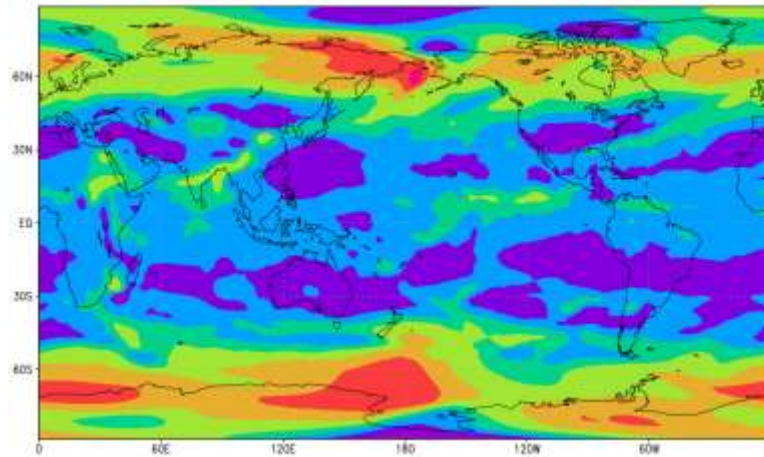
UGRD 120hr



UGRD 240hr

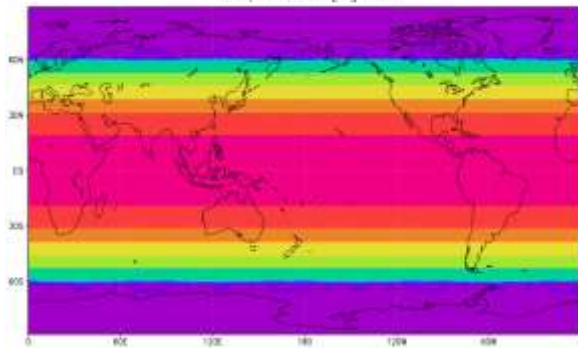


UGRD 720hr

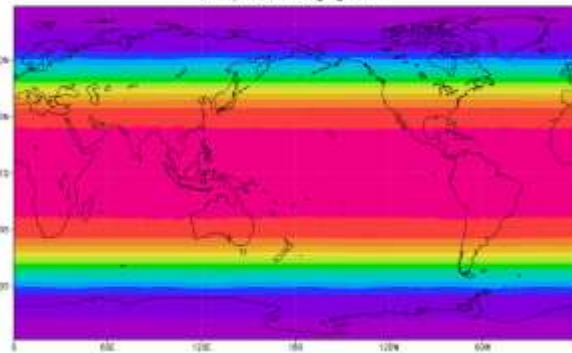


Note: we do not repeat these steps during the Training Session!

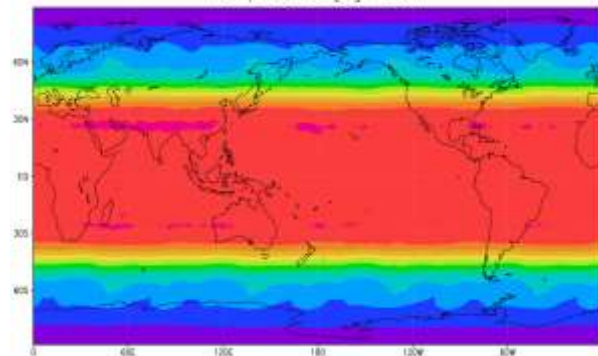
Temperature [C] 0hr



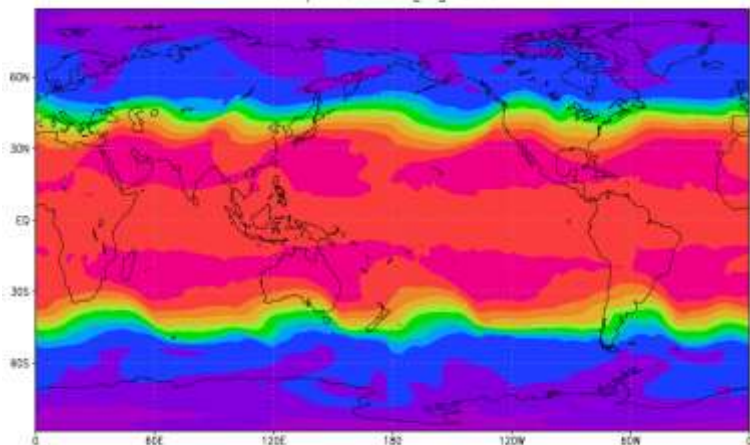
Temperature [C] 24hr



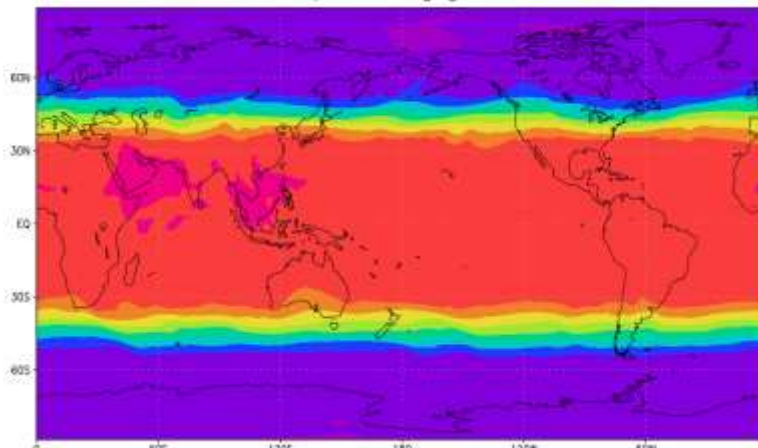
Temperature [C] 120hr



Temperature [C] 240hr

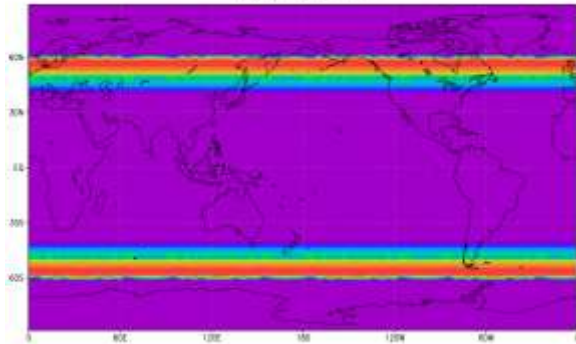


Temperature [C] 720hr

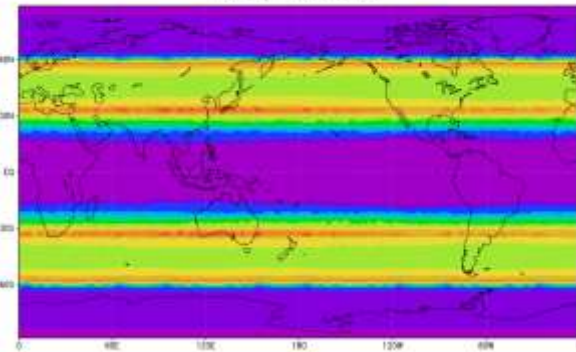


Note: we do not repeat these steps during the Training Session!

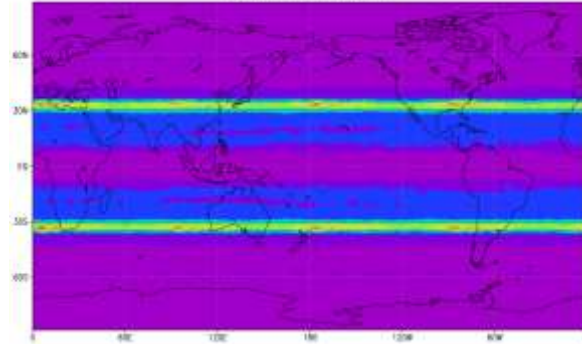
Precipitation 0hr



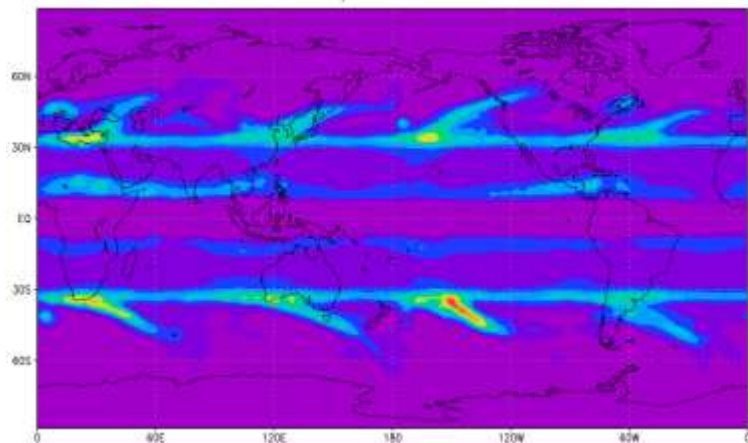
Precipitation 24hr



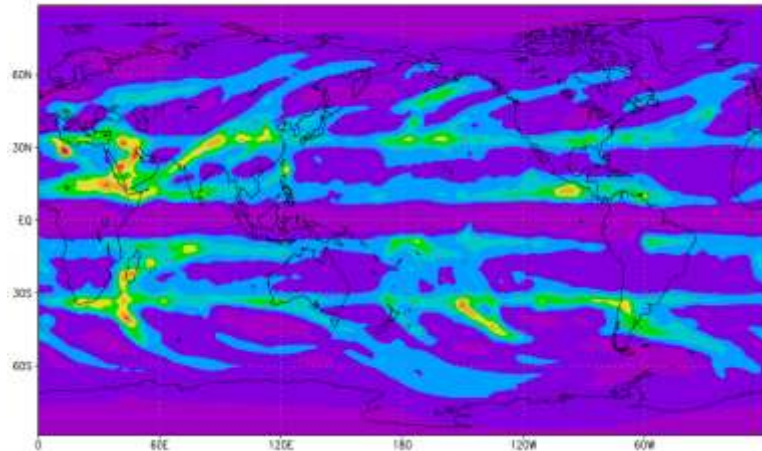
Precipitation 120hr



Precipitation 240hr



Precipitation 720hr



Note: we do not repeat these steps during the Training Session!

16. Add new files to the tests-dev directory:

```
~/ufs-weather-model/tests-dev/
```

```
| machine_config
|   └ machine_awspc.config
| test_cases
|   └ diag_table
|     └ diag_table_tc
|   └ exp_conf
|     └ 2020_CAPE.IN
|     └ aquaplanet.IN
|     └ baroclinic_wave.IN
|     └ tropical_cyclone.IN
|   └ field_table
|     └ field_table_tc
| === continued ===
```

Put your diag table
here
(if you need one)
this one is for Tropical
Cyclone case

Follow examples for other
cases and add specifics for
your test.

Put your field table here
(if you need one)



Note: we do not repeat these steps during the Training Session!

16. Add new files to the tests-dev directory:

```
| === continued ===  
|   └─ parm  
|     └─ aquaplanet.nml.IN  
|     └─ baroclinic_wave.nml.IN  
|     └─ model_configure aq.IN  
|     └─ tropical_cyclone.nml.IN  
|     └─ ufs.configure.tc.IN  
|   └─ tests  
|     └─ 2020_CAPE  
|     └─ aquaplanet  
|     └─ baroclinic_wave  
|     └─ tropical_cyclone  
|   └─ utils  
|     └─ plot_aq.sh  
|     └─ plot_tc.sh  
└─ ufs_test_utils.sh
```

Put your namelists here
(model_configure and
input.nml)

Follow examples for other cases and add
specifics for your test. This is replacing
description files from `~/ufs-weather-
model/tests/tests/*` directory.

Put plot scripts here



Note: we do not repeat these steps during the Training Session!



Exercise 1:

- 1) Run one HSD case ([aquaplanet](#))
- 2) Make changes in run directory
- 3) Submit a batch job manually (*sbatch*)
- 4) Look at the results

Run Aquaplanet HSD case

ASCII: git clone <https://github.com/NOAA-EPIC/ufs-weather-model.git> -b feature/2026-HSD-training

- Clone the code:

```
ubuntu:/>git clone https://github.com/NOAA-EPIC/ufs-weather-model -b feature/2026-HSD-training
Cloning into 'ufs-weather-model'...
remote: Enumerating objects: 33591, done.
remote: Counting objects: 100% (340/340), done.
remote: Compressing objects: 100% (161/161), done.
remote: Total 33591 (delta 242), reused 179 (delta 179), pack-reused 33251 (from 3)
Receiving objects: 100% (33591/33591), 83.60 MiB | 38.28 MiB/s, done.
Resolving deltas: 100% (27804/27804), done.
```

- Get the externals:

```
ubuntu:/>cd ufs-weather-model/
ubuntu:/>git submodule update --init --recursive
```

```
cd ufs-weather-model ; git submodule update --init --recursive
```

- Save directory path for later:

```
export SRC_DIR=`pwd`
```



Run Aquaplanet HSD case

- Run the main script (*NB: we have already launched it in the start of the Training*):

```
ubuntu:/>cd tests-dev  
ubuntu:/>./ufs_test.sh -a ubuntu -s -c -k -r -n "aquaplanet intel"
```



- Configure a cluster for a compile job: ~5 min
- Compile phase: about ~10 min
- Configure a compute node for a run job: ~ 5 min
- Run phase: about ~13 min

Run Aquaplanet HSD case

`run_dir` is a link to directory containing compilation and run directories

- Follow output files:

- While compiling

```
ubuntu:/>ll run_dir/compile_atm_dyn32_intel/out
-rw-r--r-- 1 Ratko.Vasic epic 1208867 Jun 17 2026 19:20:06 run_dir/compile_atm_dyn32_intel/out
```

- During model run

```
ubuntu:/>ll run_dir/aquaplanet_intel/out
-rw-r--r-- 1 Ratko.Vasic epic 184984 Jun 17 2026 19:26:11 run_dir/aquaplanet_intel/out
```

Run Aquaplanet HSD case

- Run is finished when you see something like this:

```
+ TEST_END_TIME='20260617 19:26:46'
+ export TEST_END_TIME
+ python3 -c 'import create_log; create_log.finish_log()'
Performing Cleanup...
REGRESSION TEST RESULT: SUCCESS
+ echo 'ufs_test.sh finished'
ufs_test.sh finished
+ cleanup
++ awk '{print $2}'
+ PID_LOCK=2883497
+ [[ 2883497 == \2\8\8\3\4\9\7 ]]
+ rm -rf /scratch4/NCEPDEV/stmp/Ratko.Vasic/ufs-weather-model/tests-dev/lock
+ [[ false == true ]]
+ trap 0
+ exit
```

Run Aquaplanet HSD case

- Check the results:

```
ubuntu: /> cd run_dir/aquaplanet_intel
ubuntu: /> ls sfcf* atmf*
atmf2166.nc  atmf2172.nc  atmf2178.nc  atmf2184.nc  sfcf2166.nc  sfcf2172.nc  sfcf2178.nc  sfcf2184.nc
```

- Save run directory in environmental variable:

```
export RUN_DIR=`pwd`
```

- Results are output every 6 hours, starting with hour 2160 (end of spin-up period)

Make changes in the run directory

- Save old results (log files, out/err files and output files):

```
ubuntu:/>mkdir orig-results-000  
ubuntu:/>mv log* atmf* sfcf* out err orig-results-000/
```

- We will use these results to compare with new, test run(s)
- Let's make changes in `input.nml`

Make changes in the run directory

```
ubuntu:/>  
ubuntu:/>vi input.nml
```

Replace horizontal advection method from 5 to 9, where:

- 5 - Fastest unlimited fifth-order scheme
- 9 - PPM with the Hunyh constraint

```
hord_mt = 5  
hord_vt = 5  
hord_tm = 5  
hord_dp = -5  
hord_tr = 8  
adjust_dry_mass = .false.  
dry_mass = 98320.0
```

```
hord_mt = 9  
hord_vt = 9  
hord_tm = 9  
hord_dp = -9  
hord_tr = 8  
adjust_dry_mass = .false.  
dry_mass = 98320.0
```


Rerun the model with changes

- After saving `input.nml` file, we can submit job using job card (`job_card`)
- Since we changed horizontal advection from fastest to more complex method, this run might run a bit slower than original. In case if job hits wallclock limit, it can be increased in `job_card`. Default is 30 minutes, so most likely we won't need it.

- Submit the job using Slurm Workflow Manager:

```
ubuntu: />  
ubuntu: /> sbatch job_card  
ubuntu: />
```

- We can check out job status using command `squeue`

```
ubuntu: />  
ubuntu: /> squeue -u $USER  
ubuntu: />
```

Check new results

- During run, we can follow err/out files with i.e. `tail -f out` until we get something like this at the end of out file:

```
0: *****RESOURCE STATISTICS*****
0: The total amount of wall time                = 544.267434
0: The total amount of time in user mode         = 267.211082
0: The total amount of time in sys mode          = 3.573101
0: The maximum resident set size (KB)           = 1617548
0: Number of page faults without I/O activity    = 2196829
0: Number of page faults with I/O activity       = 70
0: Number of times filesystem performed INPUT    = 77272
0: Number of times filesystem performed OUTPUT   = 68912
0: Number of Voluntary Context Switches         = 8025
0: Number of InVoluntary Context Switches       = 36916
0: *****END OF RESOURCE STATISTICS*****
0:
Model ended: Thu Jun 18 00:08:20 UTC 2026
```

Check new results

- And, make sure that new output files are present:

```
ubuntu:/>ll atm* sfcf*
-rw-r--r-- 1 Ratko.Vasic epic 128819405 Jun 18 2026 00:02:17 atm2166.nc
-rw-r--r-- 1 Ratko.Vasic epic 128819405 Jun 18 2026 00:04:24 atm2172.nc
-rw-r--r-- 1 Ratko.Vasic epic 128819405 Jun 18 2026 00:06:16 atm2178.nc
-rw-r--r-- 1 Ratko.Vasic epic 128819405 Jun 18 2026 00:08:19 atm2184.nc
-rw-r--r-- 1 Ratko.Vasic epic 21128927 Jun 18 2026 00:02:17 sfcf2166.nc
-rw-r--r-- 1 Ratko.Vasic epic 21128927 Jun 18 2026 00:04:25 sfcf2172.nc
-rw-r--r-- 1 Ratko.Vasic epic 21128927 Jun 18 2026 00:06:16 sfcf2178.nc
-rw-r--r-- 1 Ratko.Vasic epic 21128927 Jun 18 2026 00:08:19 sfcf2184.nc
```

Save new results

- Similar to our control run, save results (log files, out/err files and output files) to the new directory:

```
ubuntu:/>  
ubuntu:/>  
ubuntu:/>mkdir orig-results-001  
ubuntu:/>mv log* atmf* sfcf* out err orig-results-001/
```

- Now, we have control results in `orig-results-000`, and new test results in `orig-results-001` directories.

Plot Exercise 1 results

- Make sure you are in \$RUN_DIR: `cd $RUN_DIR`
- Get plot scripts from source directory:
- `cp $SRC_DIR/tests-dev/test_cases/utils/plot-test* .`

```
ubuntu:/>
ubuntu:/>ll plot-test-*
-rwxr-xr-x 1 Ratko.Vasic epic 3171 Jun 19 2026 23:07:09 plot-test-1.sh
-rwxr-xr-x 1 Ratko.Vasic epic 3185 Jun 19 2026 23:14:38 plot-test-2.sh
```

- Run plot-test-1.sh script:

```
ubuntu:/>
ubuntu:/>./plot-test-1.sh
```

Plot Exercise 1 results

- Check if all test 1 PNG files are here:

```
ubuntu:/>ll *.png
-rw-r--r-- 1 Ratko.Vasic epic 240235 Jun 19 2026 23:39:39 jet-control.png
-rw-r--r-- 1 Ratko.Vasic epic 248611 Jun 19 2026 23:39:39 jet-diff-test1.png
-rw-r--r-- 1 Ratko.Vasic epic 233466 Jun 19 2026 23:39:39 jet-test1.png
-rw-r--r-- 1 Ratko.Vasic epic 226065 Jun 19 2026 23:39:39 prec-control.png
-rw-r--r-- 1 Ratko.Vasic epic 170013 Jun 19 2026 23:39:40 prec-diff-test1.png
-rw-r--r-- 1 Ratko.Vasic epic 226078 Jun 19 2026 23:39:40 prec-test1.png
-rw-r--r-- 1 Ratko.Vasic epic 202360 Jun 19 2026 23:39:39 tmp-control.png
-rw-r--r-- 1 Ratko.Vasic epic 173741 Jun 19 2026 23:39:39 tmp-diff-test1.png
-rw-r--r-- 1 Ratko.Vasic epic 200885 Jun 19 2026 23:39:39 tmp-test1.png
```

- You can view each file using imgcat:

```
ubuntu:/>
ubuntu:/>
ubuntu:/>imgcat prec-control.png
```



Exercise 2:

- 1) Make changes in model source code
- 2) Recompile
- 3) Run experiment with the new executable
- 4) Look at the results

Make changes in the model

Make sure that we have saved environmental variables for run and model directories:

```
echo $RUN_DIR
```

```
echo $SRC_DIR
```

Now we can go to our model directory:

```
cd $SRC_DIR
```

For example, let's edit file in RRTM radiation:

```
vim UFSATM/ccpp/physics/physics/Radiation/radiation_astronomy.f
```



Make changes in the model

Make change to Solar Constant from 1370. to 1500.

```
638      solcon = solc0 / (r1*r1)
639
640 ! AQUAPLANET TEST CASE
641      solcon=1370.
642      dlt=0.
643 ! AQUAPLANET TEST CASE
644
645      slag   = sollag
646      sdec   = sindec
647      cdec   = cosdec
648
649 ! --- ... diagnostic print out
```

```
638      solcon = solc0 / (r1*r1)
639
640 ! AQUAPLANET TEST CASE
641      solcon=1500.
642      dlt=0.
643 ! AQUAPLANET TEST CASE
644
645      slag   = sollag
646      sdec   = sindec
647      cdec   = cosdec
648
649 ! --- ... diagnostic print out
```

Compile the model

- Make sure you are in \$SRC_DIR: `cd $SRC_DIR`
- Load modules:
`ubuntu:/>module use $PWD/modulefiles`
`ubuntu:/>module load ufs_awspsc.intel`
- List loaded modules:

```
ubuntu:/>module list
```

Currently Loaded Modules:

1) intel-oneapi-compilers/2024.2.1	13) libpng/1.6.37	25) sqlite/3.46.0	37) bacio/2.4.1	49) yafyaml/1.4.0
2) stack-oneapi/2024.2.1	14) hdf5/1.14.3	26) util-linux-uuid/2.40.2	38) crtm-fix/2.4.0.1_emc	50) pflogger/1.15.0
3) intel-oneapi-mpi/2021.13.1	15) snappy/1.2.1	27) python/3.11.7	39) git-lfs/3.5.1	51) udunits/2.2.28
4) stack-intel-oneapi-mpi/2021.13	16) zstd/1.5.6	28) python-venv/1.0	40) crtm/2.4.0.1	52) mapl/2.53.4-esmf-8.8.0
5) glibc/2.34	17) c-blosc/1.21.5	29) py-mpi4py/4.0.1	41) g2/3.5.1	53) scotch/7.0.4
6) intel-oneapi-runtime/2024.2.1	18) netcdf-c/4.9.2	30) intel-oneapi-mkl/2024.2.1	42) g2tmpl/1.13.0	54) ufs_common
7) nghttp/1.63.0	19) netcdf-fortran/4.6.1	31) py-numpy/1.26.4	43) ip/5.1.0	55) nccmp/1.9.1.0
8) zlib/1.2.13	20) parallel-netcdf/1.12.3	32) py-pip/23.1.2	44) sp/2.5.0	56) ufs_ursa.intel
9) curl/8.10.1	21) parallelio/2.6.2	33) libyaml/0.2.5	45) w3emc/2.10.0	
10) cmake/3.27.9	22) pigz/2.8	34) py-pyyaml/6.0.2	46) gftl/1.14.0	
11) libjpeg/2.1.0	23) tar/1.34	35) esmf/8.8.0	47) gftl-shared/1.9.0	
12) jasper/2.0.32	24) gettext/0.22.5	36) fms/2024.02	48) fargparse/1.8.0	

Compile the model

- Create build directory

- Go to build directory

```
ubuntu:/>  
ubuntu:/>mkdir build  
ubuntu:/>cd build
```

- Use cmake to create Makefile:

```
cmake .. -DAPP=ATM -DCCPP_SUITES=FV3_GFS_v17_p8_ugwpv1 -D32BIT=ON -DMPI=ON -DCMAKE_BUILD_TYPE=Release
```

- Check if Makefile is present:

```
ubuntu:/>ll Makefile  
-rw-rw-r--+ 1 Ratko.Vasic stmp 13705 Jun 18 2026 02:13:27 Makefile
```

- Run make with 6 tasks:

```
ubuntu:/>  
ubuntu:/>make -j 6
```

Compile the model

- When compilation is done, printout on screen should be like this:

```
[[100%] Linking Fortran executable ufs_model  
ifort: remark #10448: Intel(R) Fortran Compiler Classic (ifort) is now d  
using the LLVM-based Intel(R) Fortran Compiler (ifx) for continued Wind  
ag-disable=10448' to disable this message.  
ifort: command line warning #10148: option '-xopenmp' not supported  
[[100%] Built target ufs_model]
```

- Copy new executable (ufs_model) to the run directory (\$RUN_DIR)

```
ubuntu: />  
ubuntu: /> cp ufs_model $RUN_DIR/fv3.exe  
ubuntu: />
```

- Note!** During this copy command, we renamed ufs_model to fv3.exe and placed it into the run directory so job card will use it. If you want, you can save previous executable.

Rerun model with changes in the source code

- Go to the run directory:

```
ubuntu:/>  
ubuntu:/>cd $RUN_DIR
```

- In order for model to run faster, return back values for horizontal advection method in file `input.nml` from 9 to 5:

```
hord_mt = 9  
hord_vt = 9  
hord_tm = 9  
hord_dp = -9  
hord_tr = 8  
adjust_dry_mass = .false.  
dry_mass = 98320.0
```

```
hord_mt = 5  
hord_vt = 5  
hord_tm = 5  
hord_dp = -5  
hord_tr = 8  
adjust_dry_mass = .false.  
dry_mass = 98320.0
```

Rerun model with changes in the source code

- Like in Exercise 1, submit the job using Slurm Workflow Manager:

```
ubuntu:/>  
ubuntu:/> sbatch job_card  
ubuntu:/>
```

- Check out job status using command `squeue`

```
ubuntu:/>  
ubuntu:/> squeue -u $USER  
ubuntu:/>
```

Check Exercise 2 results

- Follow err/out files with i.e. `tail -f out`:
- We can also check if we use correct executable using:

`grep "SOLAR CONSTANT" out`, we should get something like this:

```
ubuntu:/>grep "SOLAR CONSTANT" out
0:  SOLAR CONSTANT      1500.0000000 (DISTANCE AJUSTED)
0:  SOLAR CONSTANT      1500.0000000 (DISTANCE AJUSTED)
0:  SOLAR CONSTANT      1500.0000000 (DISTANCE AJUSTED)
0:  SOLAR CONSTANT      1500.0000000 (DISTANCE AJUSTED)
0:  SOLAR CONSTANT      1500.0000000 (DISTANCE AJUSTED)
```

- Check if output files are done:

```
ubuntu:/>
ubuntu:/>ls atmf* sfcf*
atmf2166.nc atmf2172.nc atmf2178.nc atmf2184.nc sfcf2166.nc sfcf2172.nc sfcf2178.nc sfcf2184.nc
```

Save Exercise 2 results

- Save results to the new directory:

```
ubuntu:/>  
ubuntu:/>mkdir orig-results-002  
ubuntu:/>mv log* atmf* sfcf* out err orig-results-002/
```


Plot Exercise 2 results

- Make sure you are in \$RUN_DIR: `cd $RUN_DIR`
- You should already have plot scripts from Test 1, but if not, get it like this:
- `cp $SRC_DIR/tests-dev/test_cases/utis/plot-test* .`

```
ubuntu:/>  
ubuntu:/>ll plot-test-  
-rwxr-xr-x 1 Ratko.Vasic epic 3171 Jun 19 2026 23:07:09 plot-test-1.sh  
-rwxr-xr-x 1 Ratko.Vasic epic 3185 Jun 19 2026 23:14:38 plot-test-2.sh
```

- Run plot-test-2.sh script:

```
ubuntu:/>  
ubuntu:/>./plot-test-2.sh
```



Plot Exercise 2 results

- Plots from the Exercise 2 files:

```
ubuntu: />  
ubuntu: />ll *.png  
-rw-r--r-- 1 Ratko.Vasic epic 240235 Jun 19 2026 23:45:55 jet-control.png  
-rw-r--r-- 1 Ratko.Vasic epic 198894 Jun 19 2026 23:45:55 jet-diff-test2.png  
-rw-r--r-- 1 Ratko.Vasic epic 240465 Jun 19 2026 23:45:55 jet-test2.png  
-rw-r--r-- 1 Ratko.Vasic epic 226065 Jun 19 2026 23:45:55 prec-control.png  
-rw-r--r-- 1 Ratko.Vasic epic 165581 Jun 19 2026 23:45:56 prec-diff-test2.png  
-rw-r--r-- 1 Ratko.Vasic epic 226468 Jun 19 2026 23:45:56 prec-test2.png  
-rw-r--r-- 1 Ratko.Vasic epic 202360 Jun 19 2026 23:45:55 tmp-control.png  
-rw-r--r-- 1 Ratko.Vasic epic 198462 Jun 19 2026 23:45:55 tmp-diff-test2.png  
-rw-r--r-- 1 Ratko.Vasic epic 202376 Jun 19 2026 23:45:55 tmp-test2.png
```

- You can view each file using *imgcat*:

```
ubuntu: />  
ubuntu: />  
ubuntu: />imgcat prec-control.png
```

Questions?

- Thank you for attending!!!